COMPILERS · GROUP 5 · TEAM 06

Parser & SDT — Model Designer Design Summary

Summary of the design of the LL(1) parser and of the syntax-directed translation: what was decided, how and why.

Contents

·	Overview
1	Input: lexer tokens as terminals
2	Grammar
3	LL(1) verification
4	FIRST and FOLLOW
5	LL(1) parsing table
6	Parser: method, algorithm and trace
7	SDT design
8	SDT evaluation
9	Outputs and theoretical arguments
10	Test cases
·	References

Overview

This document summarizes the design of the syntax analyzer (parser) and the semantic analyzer (SDT) of `unam.fi.compilers.g5.06`, made by the Model Designer and implemented by the Developer in `src/parser/`. Every decision was taken with the theory seen in class only, without parsing libraries.

Design in one line: lexer tokens → 27-production LL(1) grammar → FIRST and FOLLOW → LL(1) table → predictive parser with a stack that builds the parse tree → L-attributed SDT that checks types and declarations → one of three outputs.

Assignment requirement	Section
The input is tokens from a lexical analyzer	1
A grammar whose productions match the tokens	2, 3
Top-Down or Bottom-Up parsing method	4, 5, 6
SDT rules for semantic analysis	7
Build the parse tree, then verify the SDT	6, 8
Parsing succeeds but the SDT fails	8, 9
Outputs and error in line n, with theoretical arguments	9

1. Input: lexer tokens as terminals

The parser does not read characters: it receives the tokens of the lexer from the first deliverable (unchanged). Each token becomes a terminal of the grammar:

Lexer token	Terminal
INT, FLOAT, CHAR	`int` `float` `char`
ID · INT_CONST · FLOAT_CONST · CHAR_CONST	`id` · `num` · `real` · `chr`
ASSIGN, PLUS, MINUS, STAR, SLASH, PERCENT, LPAREN, RPAREN, SEMI	`=` `+` `-` `*` `/` `%` `(` `)` `;`
Any other token (return, ==, a string…)	its own lexeme: it has no column in the table, so the parser rejects it

Example: `int x = 10;` → `int id = num ; $`. Every terminal keeps its lexeme and line for the SDT and the error messages. Constants use three terminals (`num`, `real`, `chr`) so the SDT knows their type directly. If the lexer reports an error, the token sequence is no longer the original program, so the parser is not run and the output is `Parsing error...`.

2. Grammar

The grammar extends **G1**, the left-factored grammar proposed in the lexer deliverable (`Decl → Type id Decl'`, `Decl' → ; | = Value ;`, `Type → int | float | char`, `Value → num | id`):

- `Value` was replaced by the **expression grammar seen in class** (E → TE', E' → +TE' | ε, T → FT', T' → *FT' | ε, F → id | (E)), adding `-`, `/`, `%`, `real` and `chr`. Without expressions the SDT would have almost nothing to check.
- `Program` and `StmtList` allow programs of one or more statements, so `error in line n` is meaningful.

- `Assign → id = Expr ;` allows using a variable after declaring it, which the SDT must verify.
- `Factor → - Factor` handles unary minus: the lexer design decided that constants carry no sign.

G = (N, T, P, S), with N = {Program, StmtList, Stmt, Decl, Decl', Assign, Type, Expr, Expr', Term, Term', Factor} (12), T = {`int float char id num real chr = + - * / % ( ) ;`} (16), S = Program, and P:

```
 1. Program → Stmt StmtList
 2. StmtList → Stmt StmtList
 3. StmtList → ε
 4. Stmt → Decl
 5. Stmt → Assign
 6. Decl → Type id Decl'
 7. Decl' → ;
 8. Decl' → = Expr ;
 9. Assign → id = Expr ;
10. Type → int
11. Type → float
12. Type → char
13. Expr → Term Expr'
14. Expr' → + Term Expr'
15. Expr' → - Term Expr'
16. Expr' → ε
17. Term → Factor Term'
18. Term' → * Factor Term'
19. Term' → / Factor Term'
20. Term' → % Factor Term'
21. Term' → ε
22. Factor → ( Expr )
23. Factor → - Factor
24. Factor → id
25. Factor → num
26. Factor → real
27. Factor → chr
```

Leftmost derivation of the assignment example (production numbers in parentheses): Program ⇒(1) Stmt StmtList ⇒(4) Decl StmtList ⇒(6) Type id Decl' StmtList ⇒(10) int id Decl' StmtList ⇒(8) int id = Expr ; StmtList ⇒(13)(17)(25) int id = num Term' Expr' ; StmtList ⇒(21)(16)(3) **int id = num ;**

3. LL(1) verification

- **Not ambiguous:** each precedence level is a different nonterminal (`+ -` only in Expr', `* / %` only in Term'), so `a + b * c` has a single tree.
- **No left recursion:** Expr → Expr + Term | Term was rewritten with the class rule A → Aα | β ⇒ A → βA', A' → αA' | ε. No production starts with its own head, directly or indirectly.
- **Left-factored:** the common prefix `Type id` was factored in G1 (Decl → Type id Decl'). No two alternatives share a prefix.

Formal check: for each nonterminal with several alternatives, their FIRST sets are disjoint and, if one alternative derives ε, the others do not start with a terminal of FOLLOW.

Nonterminal	Alternatives and their FIRST	FIRST sets disjoint	If an alternative derives ε	LL(1)
`StmtList`	(2) StmtList → Stmt StmtList: { int, float, char, id } (3) StmtList → ε: { ε }	yes	FOLLOW = { $ } intersection: ∅	**holds**
`Stmt`	(4) Stmt → Decl: { int, float, char } (5) Stmt → Assign: { id }	yes	does not apply	**holds**

Nonterminal	Alternatives and their FIRST	FIRST sets disjoint	If an alternative derives ε	LL(1)
`Decl'`	(7) Decl' → ;: { ; } (8) Decl' → = Expr ;: { = }	yes	does not apply	**holds**
`Type`	(10) Type → int: { int } (11) Type → float: { float } (12) Type → char: { char }	yes	does not apply	**holds**
`Expr'`	(14) Expr' → + Term Expr': { + } (15) Expr' → - Term Expr': { - } (16) Expr' → ε: { ε }	yes	FOLLOW = {), ; } intersection: ∅	**holds**
`Term'`	(18) Term' → * Factor Term': { * } (19) Term' → / Factor Term': { / } (20) Term' → % Factor Term': { % } (21) Term' → ε: { ε }	yes	FOLLOW = { +, -,), ; } intersection: ∅	**holds**
`Factor`	(22) Factor → (Expr): { (} (23) Factor → - Factor: { - } (24) Factor → id: { id } (25) Factor → num: { num } (26) Factor → real: { real } (27) Factor → chr: { chr }	yes	does not apply	**holds**

4. FIRST and FOLLOW

Computed with the class rules. **FIRST:** a terminal is its own FIRST; X → ε adds ε; for X → Y1…Yk add FIRST(Y1) − {ε}, and continue with Y2… while the previous symbols can derive ε. **FOLLOW:** `$` ∈ FOLLOW(Program); a terminal after A goes into FOLLOW(A); FIRST(B) − {ε} for a nonterminal B after A; FOLLOW of the head when A is at the end or what follows it can derive ε. Both were computed by hand and by fixed point (FIRST stabilizes after 4 passes, FOLLOW after 2), with the same result.

Nonterminal	FIRST	FOLLOW
`Program`	{ int, float, char, id }	{ $ }
`StmtList`	{ int, float, char, id, ε }	{ $ }
`Stmt`	{ int, float, char, id }	{ int, float, char, id, $ }
`Decl`	{ int, float, char }	{ int, float, char, id, $ }
`Decl'`	{ =, ; }	{ int, float, char, id, $ }
`Assign`	{ id }	{ int, float, char, id, $ }
`Type`	{ int, float, char }	{ id }
`Expr`	{ id, num, real, chr, -, (}	{), ; }
`Expr'`	{ +, -, ε }	{), ; }
`Term`	{ id, num, real, chr, -, (}	{ +, -,), ; }
`Term'`	{ *, /, %, ε }	{ +, -,), ; }
`Factor`	{ id, num, real, chr, -, (}	{ +, -, *, /, %,), ; }

Example: Term is always followed by Expr', so FOLLOW(Term) gets FIRST(Expr') − {ε} = { +, - } and, since Expr' can derive ε, also FOLLOW(Expr) = { ;,) }.

5. LL(1) parsing table

For every production A → α: it goes in M[A, a] for each a ∈ FIRST(α); if α can derive ε it goes in M[A, b] for each b ∈ FOLLOW(A). Bold numbers (yellow in the PDF) are the ε-productions placed through FOLLOW; empty cells are syntax errors.

M[A, a]	int	float	char	id	num	real	chr	=	+	-	*	/	%	(	)	;	$
Program	1	1	1	1													
StmtList	2	2	2	2													**3**
Stmt	4	4	4	5													
Decl	6	6	6														
Decl'								8								7	
Assign				9													
Type	10	11	12														
Expr				13	13	13	13			13				13			
Expr'									14	15					**16**	**16**	
Term				17	17	17	17			17				17			
Term'									**21**	**21**	18	19	20		**21**	**21**	
Factor				24	25	26	27			23				22			

51 filled cells, none with two productions: the grammar is LL(1). The `-` appears in Expr' (subtraction, 15) and in Factor (unary minus, 23) without conflict, because the nonterminal on top of the stack gives the context. The filled columns of a row are what the parser expected when a cell is empty, and that list is used in the error message. The table is delivered to the Developer as `src/parser/data/ll1_table.csv`, and a test checks that the program builds exactly the same table.

6. Parser: method, algorithm and trace

Method: non-recursive predictive Top-Down LL(1) parser (Left-to-right, Leftmost derivation, 1 lookahead token). It was chosen over Bottom-Up LR because the grammar is already LL(1), the table comes directly from FIRST and FOLLOW (12 × 17) instead of an LR automaton, the inherited attributes of the SDT are natural top-down, and the trace has the stack / input / action format used in class.

Model: input buffer (terminals + `$`), stack (initially `$ Program`, top on the right), table M[A, a] and output (productions applied). At each step, with X on top of the stack and token a:

1. X = a = `$` → **accept**.
2. X is a terminal → if X = a, pop and advance (**match**); otherwise syntax error, X was expected.
3. X is a nonterminal → if M[X, a] = X → Y1…Yk, pop X and push Yk…Y1; if the cell is empty, syntax error, a terminal of row X was expected.

Every expansion creates the children of the node, and every match stores the token (lexeme and line) in its leaf, so **the parse tree is complete when the parser accepts**; only then is the SDT evaluated. Trace of `int x = 10;` (the program prints it with `--trace`):

Step	Stack (top on the right)	Input	Action
1	`$ Program`	`int id = num ; $`	Program → Stmt StmtList
2	`$ StmtList Stmt`	`int id = num ; $`	Stmt → Decl
3	`$ StmtList Decl`	`int id = num ; $`	Decl → Type id Decl'
4	`$ StmtList Decl' id Type`	`int id = num ; $`	Type → int
5	`$ StmtList Decl' id int`	`int id = num ; $`	match int
6	`$ StmtList Decl' id`	`id = num ; $`	match id
7	`$ StmtList Decl'`	`= num ; $`	Decl' → = Expr ;
8	`$ StmtList ; Expr =`	`= num ; $`	match =
9	`$ StmtList ; Expr`	`num ; $`	Expr → Term Expr'
10	`$ StmtList ; Expr' Term`	`num ; $`	Term → Factor Term'
11	`$ StmtList ; Expr' Term' Factor`	`num ; $`	Factor → num
12	`$ StmtList ; Expr' Term' num`	`num ; $`	match num
13	`$ StmtList ; Expr' Term'`	`; $`	Term' → ε
14	`$ StmtList ; Expr'`	`; $`	Expr' → ε
15	`$ StmtList ;`	`; $`	match ;
16	`$ StmtList`	`$`	StmtList → ε
17	`$`	`$`	accept

Syntax errors. The parser stops at the first error. Example: `int x 10;` → at step 7 the top is Decl' and the token is `num`; M[Decl', num] is empty, so the message is `unexpected '10', expected '=' or ';'`. **Missing `;` rule:** if `;` is among the expected terminals and the current token is on a later line (or the input ended), the error is reported on the line of the last token read, as `missing ';' after '10'`, because that is the line where the `;` is missing.

7. SDT design

A context-free grammar cannot check that a variable is declared before use or that a value matches the variable's type: that depends on context. `int x = 3.14;` belongs to the language (`int id = real ;`) and is still wrong. The SDT attaches attributes to the symbols and semantic rules to the productions.

Attribute	Kind	Meaning
Type.type, Type.width	synthesized	int / float / char and its size (4 / 4 / 1 bytes)
Decl'.inh	inherited	declared type, from its left sibling Type
Expr.type, Term.type, Factor.type	synthesized	type of the subexpression
Expr'.inh, Term'.inh / Expr'.syn, Term'.syn	inherited / synthesized	type accumulated on the left / final type of the chain
id, num, real, chr	lexical	lexeme and line from the lexer

L-attributed: every inherited attribute depends only on the parent or on left siblings (Type → Decl', Term → Expr', Factor → Term'), so one depth-first, left-to-right traversal computes everything. Inherited attributes are required because, after removing left recursion, the left operand of `a - b` is in Term and the operator in its sibling Expr'.

Type system (an internal `error` type stops a failure from producing repeated messages):

Operation	Valid when	Result
+ - * /	both operands int or float	float if either is float, else int

Operation	Valid when	Result
%	both operands int	int
unary -	operand int or float	same type
assignment / initialization	same type, or int into float (widening)	—

`float` into `int` is rejected (it loses decimals), `char` takes no part in arithmetic and is only assigned to `char`, a variable is declared once and before use, and it is not visible in its own initializer (`int x = x;` is an error).

Symbol table: the lexer creates an entry per identifier with type `TBD`; rule 6 fills type, size and relative address, with `offset` starting at 0 and growing by `Type.width` (e.g. `int a; float b; char c; int d;` → addresses 0, 4, 8, 9). A `TBD` type means "not declared".

Semantic rules (`arith`, `neg` and `assignable` apply the type table above; `lookup` searches the symbol table):

#	Production	Semantic rule
1	`Program → Stmt StmtList`	offset := 0 at the start
2–5	`StmtList → Stmt StmtList | ε, Stmt → Decl | Assign`	— (traversal only)
6	`Decl → Type id Decl'`	Decl'.inh := Type.type. At the end: if id already has a type → error "already declared"; otherwise addtype(id, Type.type, Type.width, offset) and offset := offset + Type.width
7	`Decl' → ;`	—
8	`Decl' → = Expr ;`	if not assignable(Decl'.inh, Expr.type) → error "type mismatch"
9	`Assign → id = Expr ;`	if id has no type → error "not declared"; else if not assignable(lookup(id).type, Expr.type) → error "type mismatch"
10–12	`Type → int | float | char`	Type.type := int | float | char; Type.width := 4 | 4 | 1
13	`Expr → Term Expr'`	Expr'.inh := Term.type; Expr.type := Expr'.syn
14–15	`Expr' → + Term Expr'₁ | - Term Expr'₁`	Expr'$_1$.inh := arith(op, Expr'.inh, Term.type); Expr'.syn := Expr'$_1$.syn
16	`Expr' → ε`	Expr'.syn := Expr'.inh
17	`Term → Factor Term'`	Term'.inh := Factor.type; Term.type := Term'.syn
18–20	`Term' → * | / | % Factor Term'₁`	Term'$_1$.inh := arith(op, Term'.inh, Factor.type); Term'.syn := Term'$_1$.syn
21	`Term' → ε`	Term'.syn := Term'.inh
22	`Factor → ( Expr )`	Factor.type := Expr.type
23	`Factor → - Factor₁`	Factor.type := neg(Factor$_1$.type)
24	`Factor → id`	if id has no type → error "not declared" and Factor.type := error; else Factor.type := lookup(id).type
25–27	`Factor → num | real | chr`	Factor.type := int | float | char

Each error takes the line of its token: the `id` (rules 6, 9, 24), the `=` (type mismatch in 8 and 9) or the operator (14–20, 23).

8. SDT evaluation

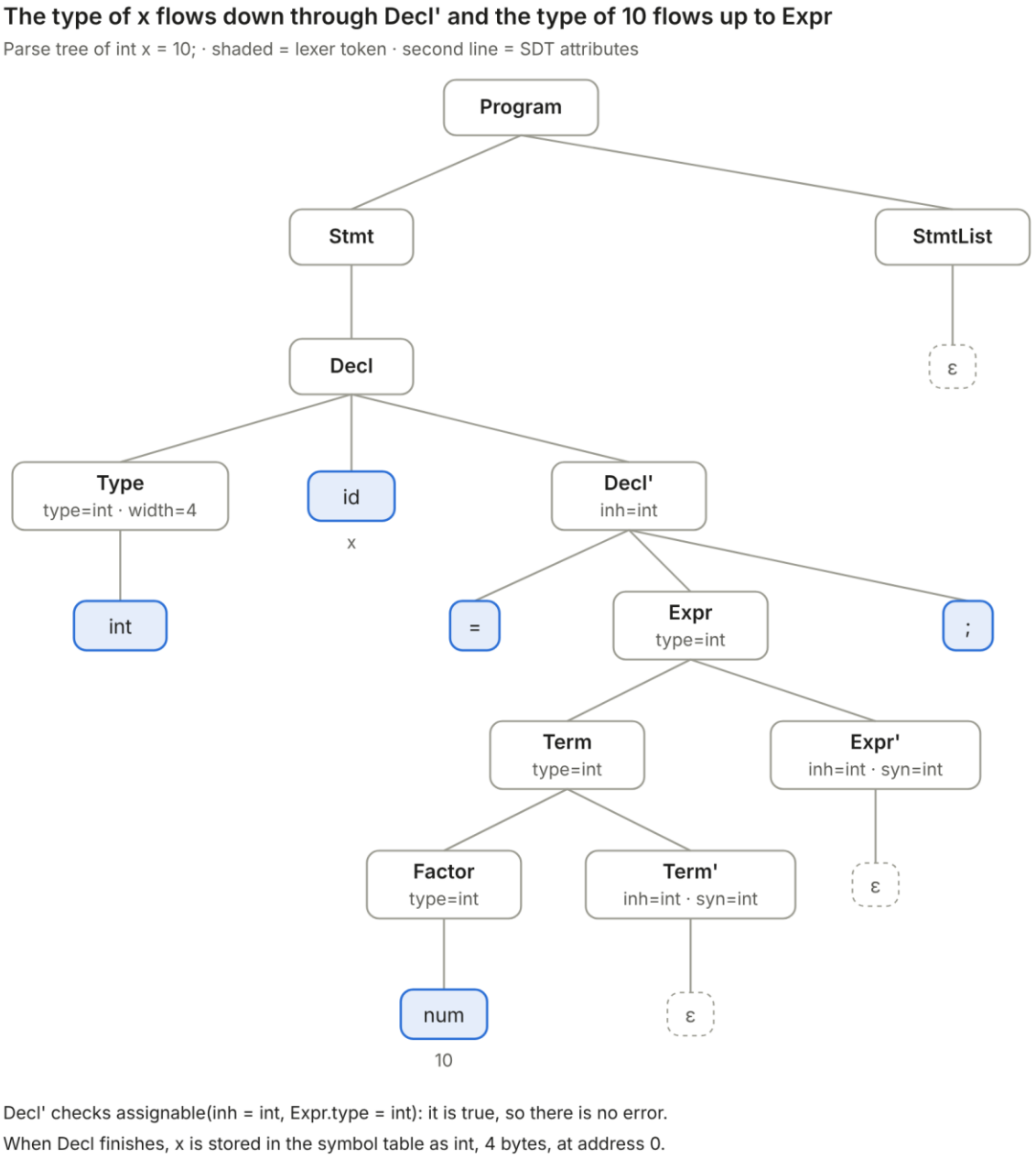


Figure 1. Annotated parse tree of int x = 10;

`int x = 10;`: Type gives int (rule 10), which flows down to Decl'.inh (6); the type of `10` flows up as int from Factor (25) through Term' and Expr' (ε, unchanged) to Expr (13). Decl' checks assignable(int, int) = true (8) and `x` is stored as int, 4 bytes, address 0.

Promotion, `float b = a * 1.5;` with `a` int: Term'.inh = int and Factor ⟨ 1.5⟩ = float, so arith(*, int, float) = float flows on to Expr; assignable(float, float) = true.

Parsing succeeds but the SDT fails: `int x = 10; float y = x * 2.5; x = y;`. Line 3 is `id = id ;`, valid syntax, but rule 9 finds lookup(x) = int and Expr.type = float, and assignable(int, float) = false → `error in line 3`.

9. Outputs and theoretical arguments

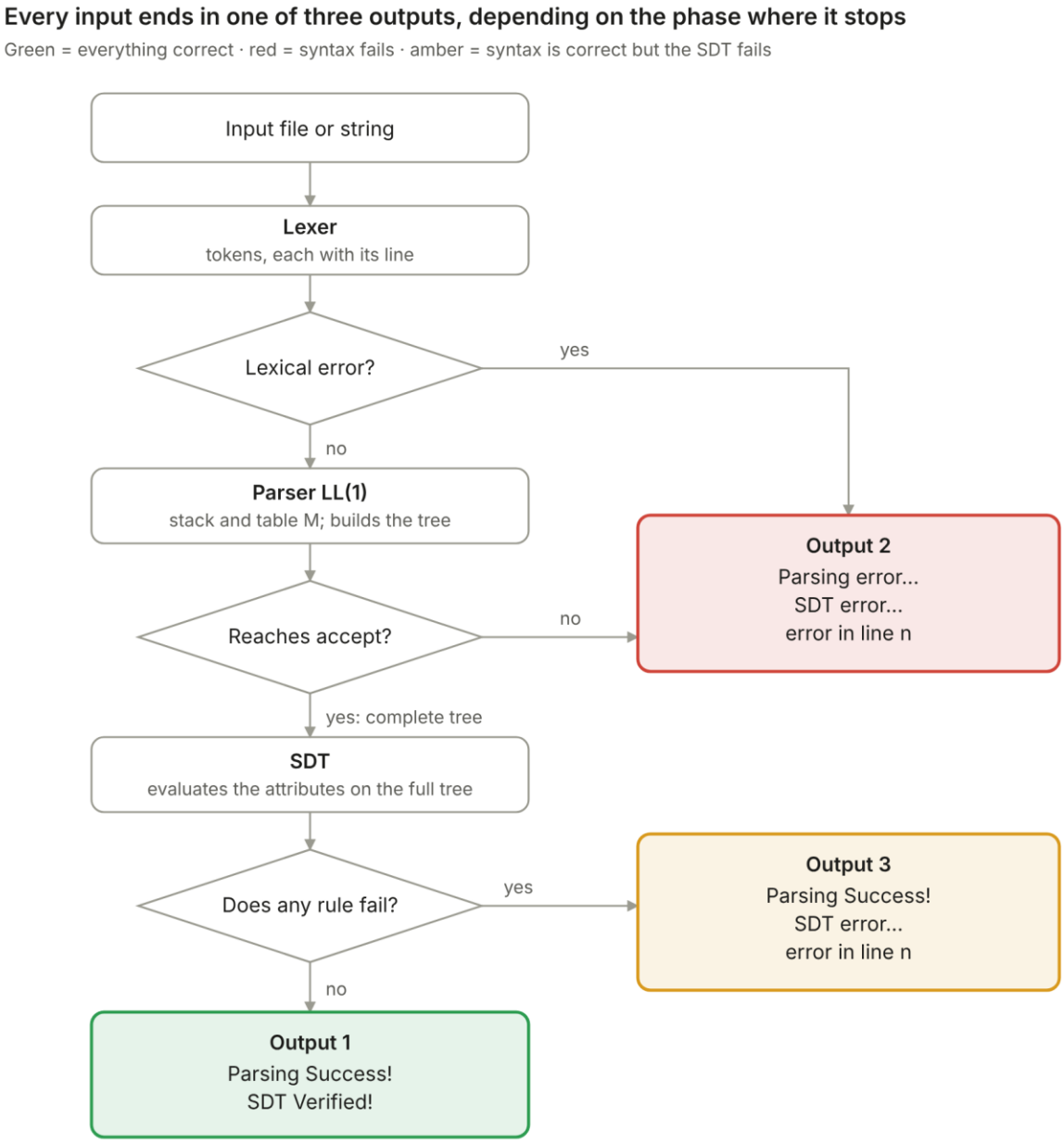


Figure 2. Every input ends in one of three outputs.

Output	Example	Theoretical argument
Parsing Success! **SDT Verified!**	`int x = 10;`	The parser reached the accepting configuration (stack $, input $); its steps form a leftmost derivation Program $\Rightarrow^*$ w, so w $\in$ L(G). All semantic rules were evaluated on the complete tree without errors: the program is well formed and well typed.
Parsing error... **SDT error...** error in line n	`int x = 10` `float y = 2.5;`	An empty cell M[A, a] or a mismatched terminal was found. Since the grammar is LL(1), the error appears at the first token that cannot continue any string of L(G) (valid prefix property). The parse tree is incomplete, so its attributes cannot be evaluated and the SDT cannot be verified. A lexical error gives this same output.

Output	Example	Theoretical argument
Parsing Success! **SDT error...** error in line n	`x = y;` with x int, y float	The input belongs to L(G), but the type of each variable depends on earlier declarations (context), which a context-free grammar cannot express. The SDT uses the symbol table and finds that a rule fails.

`error in line n`: each token carries the line assigned by the lexer, the parser stores it in the leaf and the failing rule takes it from its token. Syntax analysis stops at the first error; the SDT reports every error, because the tree is complete. Real output of `samples/parser/sdt_error.c`:

```
Input tokens:
  1: int id = num ;
  2: float id = id * real ;
  3: id = id ;
Parsing Success!
SDT error...
error in line 3: type mismatch: 'x' is int but the value is float
```

10. Test cases

Designed to check each decision; they are automated in `tests/test_parser.py` (38 tests).

Input	Expected result	Checks
`int x = 10;`	Parsing Success! · SDT Verified!	assignment example
`int x 10;`	Parsing error · unexpected '10', expected '=' or ';'	empty cell M[Decl', num]
`int x = 10` / `float y = 2.5;`	Parsing error · line 1: missing ';' after '10'	missing ; rule
`return 0;`	Parsing error · unexpected 'return'	token outside the grammar
`int @x = 10;`	Parsing error · lexical error	invalid token sequence
`float f = 10;`	Parsing Success! · SDT Verified!	int → float allowed
`int x = 3.14;`	SDT error · type mismatch	float → int rejected (rule 8)
`int x = y + 1;`	SDT error · 'y' is not declared (one message)	rule 24 and error propagation
`int x;` `int x;`	SDT error · already declared in line 1	rule 6
`float f = 2.5 % 2;`	SDT error · operator '%' needs int operands	rule 20
`char c = 'a' + 1;`	SDT error · needs numeric operands	char is not arithmetic
`int x = x;`	SDT error · 'x' is not declared	not visible in its own initializer

References

1. Assignment *Syntax & Semantic Analyzer (Parser & SDT)*, Compilers, Group 5, FI UNAM, and the class notes of the course.
2. Team 06, Lexer deliverable (grammar G1, token catalog, symbol table).
3. Aho, A. V., Lam, M. S., Sethi, R. and Ullman, J. D. (2006). *Compilers: Principles, Techniques, and Tools* (2nd ed.). Addison-Wesley. Ch. 4–6.
4. Louden, K. C. (1997). *Compiler Construction: Principles and Practice*. PWS Publishing.